Concepts

Orders

NautilusTrader supports a broad set of order types and execution instructions, exposing as much of a trading venue's functionality as possible. Traders can define instructions and contingencies for order execution and management across any trading strategy.

Overview

All order types are derived from two fundamentals: *Market* and *Limit* orders. In terms of liquidity, they are opposites. *Market* orders consume liquidity by executing immediately at the best available price, whereas *Limit* orders provide liquidity by resting in the order book at a specified price until matched.

The order types available for the platform are (using the `OrderType` enum values):

- `MARKET`
- `LIMIT`
- `STOP_MARKET`
- `STOP_LIMIT`
- `MARKET_TO_LIMIT`
- `MARKET_IF_TOUCHED`
- `LIMIT_IF_TOUCHED`
- `TRAILING_STOP_MARKET`
- `TRAILING_STOP_LIMIT`

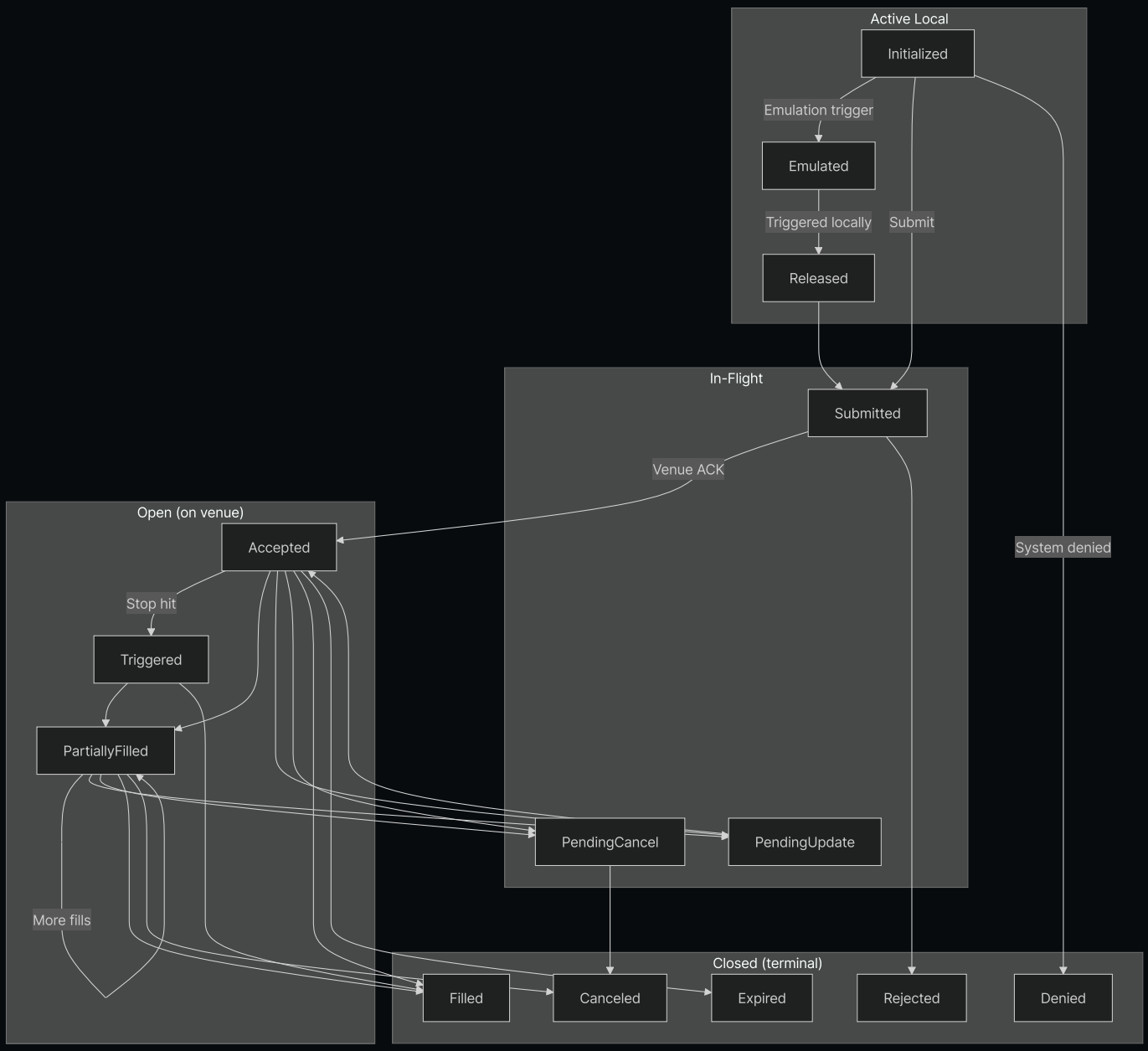
> NautilusTrader provides a unified API for many order types and execution instructions, but not all venues support every option. If an order includes an instruction or option the target venue does not support, the system does not submit it. Instead, it logs a clear, explanatory error.

Terminology

- An order is **aggressive** if its type is `MARKET` or if it executes as a *marketable* order (i.e., takes liquidity).
- An order is **passive** if it is not marketable (i.e., provides liquidity).
- An order is **active local** if it remains within the local system boundary in one of the following three non-terminal statuses:
  - `INITIALIZED`
  - `EMULATED`
  - `RELEASED`
- An order is **in-flight** when at one of the following statuses:
  - `SUBMITTED`
  - `PENDING_UPDATE`
  - `PENDING_CANCEL`
- An order is **open** when at one of the following (non-terminal) statuses:
  - `ACCEPTED`
  - `TRIGGERED`
  - `PENDING_UPDATE`
  - `PENDING_CANCEL`
  - `PARTIALLY_FILLED`
- An order is **closed** when at one of the following (terminal) statuses:
  - `DENIED`
  - `REJECTED`
  - `CANCELED`
  - `EXPIRED`
  - `FILLED`

Order state flow

The following diagram illustrates the order lifecycle and primary state transitions:



Order status definitions

Status	Description
`INITIALIZED`	Order is instantiated within the Nautilus system.
`DENIED`	Order was denied by Nautilus for being invalid, unprocessable, or exceeding a risk limit.
`EMULATED`	Order is being emulated by the `OrderEmulator` component.
`RELEASED`	Order was released from the `OrderEmulator` component.
`SUBMITTED`	Order was submitted to the venue (awaiting acknowledgement).

Status	Description
`ACCEPTED`	Order was acknowledged by the venue as received and valid (may now be working).
`REJECTED`	Order was rejected by the trading venue.
`CANCELED`	Order was canceled (terminal).
`EXPIRED`	Order reached its GTD expiration (terminal).
`TRIGGERED`	Order's STOP price was triggered on the venue.
`PENDING_UPDATE`	Order is pending a modification request on the venue.
`PENDING_CANCEL`	Order is pending a cancellation request on the venue.
`PARTIALLY_FILLED`	Order has been partially filled on the venue.
`FILLED`	Order has been completely filled (terminal).

Execution instructions

Certain venues allow a trader to specify conditions and restrictions on how an order will be processed and executed. The following is a brief summary of the different execution instructions available.

Time in force

The order's time in force specifies how long the order will remain open or active before any remaining quantity is canceled.

- `GTC` **(Good Till Cancel)**: The order remains active until canceled by the trader or the venue.
- `IOC` **(Immediate or Cancel / Fill and Kill)**: The order executes immediately, with any unfilled portion canceled.
- `FOK` **(Fill or Kill)**: The order executes immediately in full or not at all.
- `GTD` **(Good Till Date)**: The order remains active until a specified expiration date and time.

- `DAY` (Good for session/day): The order remains active until the end of the current trading session.
- `AT_THE_OPEN` (OPG): The order is only active at the open of the trading session.
- `AT_THE_CLOSE`: The order is only active at the close of the trading session.

Expire time

This instruction is to be used in conjunction with the `GTD` time in force to specify the time at which the order will expire and be removed from the venue's order book (or order management system).

Post-only

An order which is marked as `post_only` will only ever participate in providing liquidity to the limit order book, and never initiating a trade which takes liquidity as an aggressor. This option is important for market makers, or traders seeking to restrict the order to a liquidity *maker* fee tier.

Reduce-only

An order which is set as `reduce_only` will only ever reduce an existing position on an instrument and never open a new position (if already flat). The exact behavior of this instruction can vary between venues.

However, the behavior in the Nautilus `SimulatedExchange` is typical of a real venue.

- Order will be canceled if the associated position is closed (becomes flat).
- Order quantity will be reduced as the associated position's size decreases.

Display quantity

The `display_qty` specifies the portion of a *Limit* order which is displayed on the limit order book. These are also known as iceberg orders as there is a visible portion to be displayed, with more quantity which is hidden. Specifying a display quantity of zero is also equivalent to setting an order as `hidden`.

Trigger type

Also known as trigger method which is applicable to conditional trigger orders, specifying the method of triggering the stop price.

- `DEFAULT`: The default trigger type for the venue (typically `LAST_PRICE` or `BID_ASK`).
- `LAST_PRICE`: The trigger price will be based on the last traded price.
- `BID_ASK`: The trigger price will be based on the bid for buy orders and ask for sell orders.
- `DOUBLE_LAST`: The trigger price will be based on the last two consecutive last prices.
- `DOUBLE_BID_ASK`: The trigger price will be based on the last two consecutive bid or ask prices as applicable.
- `LAST_OR_BID_ASK`: The trigger price will be based on either the last price or bid/ask.
- `MID_POINT`: The trigger price will be based on the mid-point between the bid and ask.
- `MARK_PRICE`: The trigger price will be based on the venue's mark price for the instrument.
- `INDEX_PRICE`: The trigger price will be based on the venue's index price for the instrument.

Trigger offset type

Applicable to conditional trailing-stop trigger orders, specifies the method of triggering modification of the stop price based on the offset from the *market* (bid, ask or last price as applicable).

- `DEFAULT`: The default offset type for the venue (typically `PRICE`).
- `PRICE`: The offset is based on a price difference.
- `BASIS_POINTS`: The offset is based on a price percentage difference expressed in basis points (100bp = 1%).
- `TICKS`: The offset is based on a number of ticks.
- `PRICE_TIER`: The offset is based on a venue-specific price tier.

Contingent orders

More advanced relationships can be specified between orders. For example, child orders can be assigned to trigger only when the parent is activated or filled, or orders can be linked so that one cancels or reduces the quantity of another. See the Advanced Orders section for more details.

Order factory

The easiest way to create new orders is by using the built-in `OrderFactory`, which is automatically attached to every `Strategy` class. This factory will take care of lower level details - such as ensuring the correct trader ID and strategy ID are assigned, generation of a necessary initialization ID and timestamp, and abstracts away parameters which don't necessarily apply to the order type being created, or are only needed to specify more advanced execution instructions.

This leaves the factory with simpler order creation methods to work with, all the examples use an `OrderFactory` from within a `Strategy` context.

See the `OrderFactory` API Reference for further details.

Order types

The following describes the order types which are available for the platform with a code example. Any optional parameters will be clearly marked with a comment which includes the default value.

Market

A *Market* order is an instruction by the trader to immediately trade the given quantity at the best price available. You can also specify several time in force options, and indicate whether this order is only intended to reduce a position.

In the following example we create a *Market* order on the Interactive Brokers IdealPro Forex ECN to BUY 100,000 AUD using USD:

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import MarketOrder

order: MarketOrder = self.order_factory.market(
    instrument_id=InstrumentId.from_str("AUD/USD.IDEALPRO"),
    order_side=OrderSide.BUY,
    quantity=Quantity.from_int(100_000),
    time_in_force=TimeInForce.IOC,  # <-- optional (default GTC)
    reduce_only=False,  # <-- optional (default False)
```

```
    tags=["ENTRY"],  # <-- optional (default None)
)
```

See the `MarketOrder` API Reference for further details.

Limit

A *Limit* order is placed on the limit order book at a specific price, and will only execute at that price (or better).

In the following example we create a *Limit* order on the Binance Futures Crypto exchange to SELL 20 ETHUSDT-PERP Perpetual Futures contracts at a limit price of 5000 USDT, as a market maker.

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import LimitOrder

order: LimitOrder = self.order_factory.limit(
    instrument_id=InstrumentId.from_str("ETHUSDT-PERP.BINANCE"),
    order_side=OrderSide.SELL,
    quantity=Quantity.from_int(20),
    price=Price.from_str("5_000.00"),
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    expire_time=None,  # <-- optional (default None)
    post_only=True,  # <-- optional (default False)
    reduce_only=False,  # <-- optional (default False)
    display_qty=None,  # <-- optional (default None which indicates full display)
    tags=None,  # <-- optional (default None)
)
```

See the `LimitOrder` API Reference for further details.

Stop-Market

A *Stop-Market* order is a conditional order which once triggered, will immediately place a *Market* order. This order type is often used as a stop-loss to limit losses, either as a SELL order against LONG positions, or as a BUY order against SHORT positions.

In the following example we create a *Stop-Market* order on the Binance Spot/Margin exchange to SELL 1 BTC at a trigger price of 100,000 USDT, active until further notice:

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import StopMarketOrder

order: StopMarketOrder = self.order_factory.stop_market(
    instrument_id=InstrumentId.from_str("BTCUSDT.BINANCE"),
    order_side=OrderSide.SELL,
    quantity=Quantity.from_int(1),
    trigger_price=Price.from_int(100_000),
    trigger_type=TriggerType.LAST_PRICE,  # <-- optional (default DEFAULT)
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    expire_time=None,  # <-- optional (default None)
    reduce_only=False,  # <-- optional (default False)
    tags=None,  # <-- optional (default None)
)
```

See the `StopMarketOrder` API Reference for further details.

Stop-Limit

A *Stop-Limit* order is a conditional order which once triggered will immediately place a *Limit* order at the specified price.

In the following example we create a *Stop-Limit* order on the Currenex FX ECN to BUY 50,000 GBP at a limit price of 1.3000 USD once the market hits the trigger price of 1.30010 USD, active until midday 6th June, 2022 (UTC):

```
import pandas as pd
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import StopLimitOrder

order: StopLimitOrder = self.order_factory.stop_limit(
    instrument_id=InstrumentId.from_str("GBP/USD.CURRENEX"),
    order_side=OrderSide.BUY,
    quantity=Quantity.from_int(50_000),
    price=Price.from_str("1.30000"),
    trigger_price=Price.from_str("1.30010"),
    trigger_type=TriggerType.BID_ASK,  # <-- optional (default DEFAULT)
    time_in_force=TimeInForce.GTD,  # <-- optional (default GTC)
    expire_time=pd.Timestamp("2022-06-06T12:00"),
```

```
    post_only=True,  # <-- optional (default False)
    reduce_only=False,  # <-- optional (default False)
    tags=None,  # <-- optional (default None)
)
```

See the `StopLimitOrder` API Reference for further details.

Market-To-Limit

A *Market-To-Limit* order submits as a market order at the current best price. If the order partially fills, the system cancels the remainder and resubmits it as a *Limit* order at the executed price.

In the following example we create a *Market-To-Limit* order on the Interactive Brokers IdealPro Forex ECN to BUY 200,000 USD using JPY:

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import MarketToLimitOrder

order: MarketToLimitOrder = self.order_factory.market_to_limit(
    instrument_id=InstrumentId.from_str("USD/JPY.IDEALPRO"),
    order_side=OrderSide.BUY,
    quantity=Quantity.from_int(200_000),
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    reduce_only=False,  # <-- optional (default False)
    display_qty=None,  # <-- optional (default None which indicates full display)
    tags=None,  # <-- optional (default None)
)
```

See the `MarketToLimitOrder` API Reference for further details.

Market-If-Touched

A *Market-If-Touched* order is a conditional order which once triggered will immediately place a *Market* order. This order type is often used to enter a new position on a stop price, or to take profits for an existing position, either as a SELL order against LONG positions, or as a BUY order against SHORT positions.

In the following example we create a *Market-If-Touched* order on the Binance Futures exchange to SELL 10 ETHUSDT-PERP Perpetual Futures contracts at a trigger price of 10,000 USDT, active until further notice:

```
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import MarketIfTouchedOrder

order: MarketIfTouchedOrder = self.order_factory.market_if_touched(
    instrument_id=InstrumentId.from_str("ETHUSDT-PERP.BINANCE"),
    order_side=OrderSide.SELL,
    quantity=Quantity.from_int(10),
    trigger_price=Price.from_str("10_000.00"),
    trigger_type=TriggerType.LAST_PRICE,  # <-- optional (default DEFAULT)
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    expire_time=None,  # <-- optional (default None)
    reduce_only=False,  # <-- optional (default False)
    tags=["ENTRY"],  # <-- optional (default None)
)
```

See the `MarketIfTouchedOrder` API Reference for further details.

Limit-If-Touched

A *Limit-If-Touched* order is a conditional order which once triggered will immediately place a *Limit* order at the specified price.

In the following example we create a *Limit-If-Touched* order to BUY 5 BTCUSDT-PERP Perpetual Futures contracts on the Binance Futures exchange at a limit price of 30,100 USDT (once the market hits the trigger price of 30,150 USDT), active until midday 6th June, 2022 (UTC):

```
import pandas as pd
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import LimitIfTouchedOrder

order: LimitIfTouchedOrder = self.order_factory.limit_if_touched(
    instrument_id=InstrumentId.from_str("BTCUSDT-PERP.BINANCE"),
    order_side=OrderSide.BUY,
    quantity=Quantity.from_int(5),
    price=Price.from_str("30_100"),
    trigger_price=Price.from_str("30_150"),
    trigger_type=TriggerType.LAST_PRICE,  # <-- optional (default DEFAULT)
    time_in_force=TimeInForce.GTD,  # <-- optional (default GTC)
```

```
    expire_time=pd.Timestamp("2022-06-06T12:00"),
    post_only=True,  # <-- optional (default False)
    reduce_only=False,  # <-- optional (default False)
    tags=["TAKE_PROFIT"],  # <-- optional (default None)
)
```

See the `LimitIfTouchedOrder` API Reference for further details.

Trailing-Stop-Market

A *Trailing-Stop-Market* order is a conditional order which trails a stop trigger price a fixed offset away from the defined market price. Once triggered a *Market* order will immediately be placed.

In the following example we create a *Trailing-Stop-Market* order on the Binance Futures exchange to SELL 10 ETHUSD-PERP COIN_M margined Perpetual Futures Contracts activating at a price of 5,000 USD, then trailing at an offset of 1% (in basis points) away from the current last traded price:

```
import pandas as pd
from decimal import Decimal
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model.enums import TrailingOffsetType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import TrailingStopMarketOrder

order: TrailingStopMarketOrder = self.order_factory.trailing_stop_market(
    instrument_id=InstrumentId.from_str("ETHUSD-PERP.BINANCE"),
    order_side=OrderSide.SELL,
    quantity=Quantity.from_int(10),
    activation_price=Price.from_str("5_000"),
    trigger_type=TriggerType.LAST_PRICE,  # <-- optional (default DEFAULT)
    trailing_offset=Decimal(100),
    trailing_offset_type=TrailingOffsetType.BASIS_POINTS,
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    expire_time=None,  # <-- optional (default None)
    reduce_only=True,  # <-- optional (default False)
    tags=["TRAILING_STOP-1"],  # <-- optional (default None)
)
```

See the `TrailingStopMarketOrder` API Reference for further details.

Trailing-Stop-Limit

A *Trailing-Stop-Limit* order is a conditional order which trails a stop trigger price a fixed offset away from the defined market price. Once triggered a *Limit* order will immediately be placed at the defined price (which is also updated as the market moves until triggered).

In the following example we create a *Trailing-Stop-Limit* order on the Currenex FX ECN to BUY 1,250,000 AUD using USD at a limit price of 0.71000 USD, activating at 0.72000 USD then trailing at a stop offset of 0.00100 USD away from the current ask price, active until further notice:

```
import pandas as pd
from decimal import Decimal
from nautilus_trader.model.enums import OrderSide
from nautilus_trader.model.enums import TimeInForce
from nautilus_trader.model.enums import TriggerType
from nautilus_trader.model.enums import TrailingOffsetType
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Price
from nautilus_trader.model import Quantity
from nautilus_trader.model.orders import TrailingStopLimitOrder

order: TrailingStopLimitOrder = self.order_factory.trailing_stop_limit(
    instrument_id=InstrumentId.from_str("AUD/USD.CURRENEX"),
    order_side=OrderSide.BUY,
    quantity=Quantity.from_int(1_250_000),
    price=Price.from_str("0.71000"),
    activation_price=Price.from_str("0.72000"),
    trigger_type=TriggerType.BID_ASK,  # <-- optional (default DEFAULT)
    limit_offset=Decimal("0.00050"),
    trailing_offset=Decimal("0.00100"),
    trailing_offset_type=TrailingOffsetType.PRICE,
    time_in_force=TimeInForce.GTC,  # <-- optional (default GTC)
    expire_time=None,  # <-- optional (default None)
    reduce_only=True,  # <-- optional (default False)
    tags=["TRAILING_STOP"],  # <-- optional (default None)
)
```

See the `TrailingStopLimitOrder` API Reference for further details.

Advanced orders

The following guide should be read in conjunction with the specific documentation from the broker or venue involving these order types, lists/groups and execution instructions (such as for Interactive Brokers).

Order lists

Combinations of contingent orders, or larger order bulks can be grouped together into a list with a common `order_list_id`. The orders contained in this list may or may not have a contingent relationship with each other, as this is specific to how the orders themselves are constructed, and the specific venue they are being routed to.

Contingency types

- **OTO (One-Triggers-Other)** – a parent order that, once executed, automatically places one or more child orders.
  - *Full-trigger model*: child order(s) are released **only after the parent is completely filled**. Common at most retail equity/option brokers (e.g. Schwab, Fidelity, TD Ameritrade) and many spot-crypto venues (Binance, Coinbase).
  - *Partial-trigger model*: child order(s) are released **pro-rata to each partial fill**. Used by professional-grade platforms such as Interactive Brokers, most futures/FX OMSs, and Kraken Pro.
- **OCO (One-Cancels-Other)** – two (or more) linked live orders where executing one cancels the remainder.
- **OUO (One-Updates-Other)** – two (or more) linked live orders where executing one reduces the open quantity of the remainder.

> These contingency types relate to ContingencyType FIX tag <1385> https://www.onixs.biz/fix-dictionary/5.0.sp2/tagnum_1385.html.

One-Triggers-Other (OTO)

An OTO order involves two parts:

1. **Parent order** – submitted to the matching engine immediately.
2. **Child order(s)** – held *off-book* until the trigger condition is met.

Trigger models

Trigger model	When are child orders released?
Full trigger	When the parent order's cumulative quantity equals its original quantity (i.e., it is *fully* filled).
Partial trigger	Immediately upon each partial execution of the parent; the child's quantity matches the executed amount and is increased as further fills occur.

> The default backtest venue for NautilusTrader uses a *partial-trigger model* for OTO orders. To opt-in to a *full-trigger mode*, set `oto_trigger_mode="FULL"` for the venue (e.g. via `BacktestVenueConfig`).

Working with partial-trigger in production:

If your strategy requires full-trigger semantics but the venue or backtest engine uses partial-trigger:

1. Submit the parent order without contingent children.
2. Subscribe to `OrderFilled` events for the parent order.
3. Only submit child orders (stop-loss, take-profit) after confirming the parent is fully filled.
4. Use `order.is_closed` and `order.filled_qty == order.quantity` to verify complete fill.

> *"Why the distinction matters Full trigger leaves a risk window: any partially filled position is live without its protective exit until the remaining quantity fills. Partial trigger mitigates that risk by ensuring every executed lot instantly has its linked stop/limit, at the cost of creating more order traffic and updates."*

An OTO order can use any supported asset type on the venue (e.g. stock entry with option hedge, futures entry with OCO bracket, crypto spot entry with TP/SL).

Venue / Adapter ID	Asset classes	Trigger rule for child	Practical notes
Binance / Binance Futures (`BINANCE`)	Spot, perpetual futures	**Partial or full** – fires on first fill.	OTOCO/TP-SL children appear instantly; monitor margin usage.

Venue / Adapter ID	Asset classes	Trigger rule for child	Practical notes
Bybit Spot (`BYBIT`)	Spot	**Full** – child placed after completion.	TP-SL preset activates only once the limit order is fully filled.
Bybit Perps (`BYBIT`)	Perpetual futures	**Partial and full** – configurable.	“Partial-position” mode sizes TP-SL as fills arrive.
Kraken Futures (`KRAKEN`)	Futures & perps	**Partial and full** – automatic.	Child quantity matches every partial execution.
OKX (`OKX`)	Spot, futures, options	**Full** – attached stop waits for fill.	Position-level TP-SL can be added separately.
Interactive Brokers (`INTERACTIVE_BROKERS`)	Stocks, options, FX, fut	**Configurable** – OCA can pro-rate.	`OcaType 2/3` reduces remaining child quantities.
dYdX v4 (`DYDX`)	Perpetual futures (DEX)	On-chain condition (size exact).	TP-SL triggers by oracle price; partial fill not applicable.
Polymarket (`POLYMARKET`)	Prediction market (DEX)	N/A.	Advanced contingency handled entirely at the strategy layer.
Betfair (`BETFAIR`)	Sports betting	N/A.	Advanced contingency handled entirely at the strategy layer.

One-Cancels-Other (OCO)

An OCO order is a set of linked orders where the execution of **any** order (full *or partial*) triggers a best-efforts cancellation of the others. Both orders are live simultaneously; once one starts filling, the venue attempts to cancel the unexecuted portion of the remainder.

One-Updates-Other (OUO)

An OUO order is a set of linked orders where execution of one order causes an immediate *reduction* of open quantity in the other order(s). Both orders are live concurrently, and each partial execution proportionally updates the remaining quantity of its peer order on a best-effort basis.

Contingent order validation

When working with contingent orders (OTO, OCO, OUO), be aware of the following validation rules and error scenarios:

Order list requirements:

- All orders in a contingent group must share the same `order_list_id`.
- Parent orders must be submitted before or simultaneously with their children.
- Child orders reference their parent via `parent_order_id`.

Modification rules:

- Parent orders can typically be modified while pending, but modifications may cascade to children.
- Child orders can be modified independently on most venues, but check venue-specific behavior.
- Canceling a parent order will cancel all associated child orders.

Common error scenarios:

Scenario	System behavior
Child references non-existent parent	Order denied with `INVALID_ORDER` error
Parent canceled before children trigger	Children automatically canceled
OCO sibling filled before cancel propagates	Partial fill honored, remaining quantity canceled
Insufficient margin for bracket	Entry may execute, children rejected separately

> ⚠️ Always handle `OrderDenied` and `OrderRejected` events in your strategy, especially for contingent orders where partial failures can leave positions unprotected.

Bracket orders

Bracket orders are an advanced order type that allows traders to set both take-profit and stop-loss levels for a position simultaneously. This involves placing a parent order (entry order) and two child orders: a take-profit `LIMIT` order and a stop-loss `STOP_MARKET` order. When

the parent order executes, the system places the child orders. The take-profit closes the position if the market moves favorably, and the stop-loss limits losses if it moves unfavorably.

Bracket orders can be easily created using the OrderFactory, which supports various order types, parameters, and instructions.

> ⚠ You should be aware of the margin requirements of positions, as bracketing a position will consume more order margin.

Emulated orders

Introduction

Emulation lets you use order types even when your trading venue does not natively support them.

Nautilus locally mimics the behavior of these order types (such as `STOP_LIMIT` or `TRAILING_STOP` orders) while using only `MARKET` and `LIMIT` orders for actual execution on the venue.

When you create an emulated order, Nautilus continuously tracks a specific type of market price (specified by the `emulation_trigger` parameter) and based on the order type and conditions you've set, will automatically submit the appropriate fundamental order (`MARKET` / `LIMIT`) when the triggering condition is met.

For example, if you create an emulated `STOP_LIMIT` order, Nautilus will monitor the market price until your `stop` price is reached, and then automatically submits a `LIMIT` order to the venue.

To perform emulation, Nautilus needs to know which **type of market price** it should monitor. By default, it uses bid and ask prices (quotes), which is why you'll often see `emulation_trigger=TriggerType.DEFAULT` in examples (this is equivalent to using `TriggerType.BID_ASK`). However, Nautilus supports various other price types, that can guide the emulation process.

Submitting an order for emulation

The only requirement to emulate an order is to pass a `TriggerType` to the `emulation_trigger` parameter of an `Order` constructor, or `OrderFactory` creation method. The following emulation

trigger types are currently supported:

- `NO_TRIGGER`: disables local emulation completely and order is fully submitted to the venue.
- `DEFAULT`: which is the same as `BID_ASK`.
- `BID_ASK`: emulated using quotes to trigger.
- `LAST_PRICE`: emulated using trades to trigger.

The choice of trigger type determines how the order emulation will behave:

- For `STOP` orders, the trigger price will be compared against the specified trigger type.
- For `TRAILING_STOP` orders, the trailing offset will be updated based on the specified trigger type.
- For `LIMIT` orders being emulated, the limit price will be compared against the specified trigger type to determine when to release the order as a `MARKET` order.

Here are all the available values you can set into `emulation_trigger` parameter and their purposes:

Trigger Type	Description	Common use cases
`NO_TRIGGER`	Disables emulation completely. The order is sent directly to the venue without any local processing.	When you want to use the venue's native order handling, or for simple order types that don't need emulation.
`DEFAULT`	Same as `BID_ASK`. This is the standard choice for most emulated orders.	General-purpose emulation when you want to work with the "default" type of market prices.
`BID_ASK`	Uses the best bid and ask prices (quotes) to guide emulation.	Stop orders, trailing stops, and other orders that should react to the current market spread.
`LAST_PRICE`	Uses the price of the most recent trade to guide emulation.	Orders that should trigger based on actual executed trades rather than quotes.
`DOUBLE_LAST`	Uses two consecutive last trade prices to confirm the trigger condition.	When you want additional confirmation of price movement before triggering.

Trigger Type	Description	Common use cases
`DOUBLE_BID_ASK`	Uses two consecutive bid/ask price updates to confirm the trigger condition.	When you want extra confirmation of quote movements before triggering.
`LAST_OR_BID_ASK`	Triggers on either last trade price or bid/ask prices.	When you want to be more responsive to any type of price movement.
`MID_POINT`	Uses the middle point between the best bid and ask prices.	Orders that should trigger based on the theoretical fair price.
`MARK_PRICE`	Uses the mark price (common in derivatives markets) for triggering.	Particularly useful for futures and perpetual contracts.
`INDEX_PRICE`	Uses an underlying index price for triggering.	When trading derivatives that track an index.

Technical details

The platform makes it possible to emulate most order types locally, regardless of whether the type is supported on a trading venue. The logic and code paths for order emulation are exactly the same for all environment contexts and use a common `OrderEmulator` component.

> There is no limitation on the number of emulated orders you can have per running instance.

Lifecycle

An emulated order will progress through the following stages:

1. Submitted by a `Strategy` through the `submit_order` method.
2. Sent to the `RiskEngine` for pre-trade risk checks (it may be denied at this point).
3. Sent to the `OrderEmulator` where it is *held* / emulated.
4. Once triggered, emulated order is transformed into a `MARKET` or `LIMIT` order and released (submitted to the venue).
5. Released order undergoes final risk checks before venue submission.

> Emulated orders are subject to the same risk controls as *regular* orders, and can be modified and canceled by a trading strategy in the normal way. They will also be included when canceling all

orders.

An emulated order will retain its original client order ID throughout its entire life cycle, making it easy to query through the cache.

Held emulated orders

The following will occur for an emulated order now *held* by the `OrderEmulator` component:

- The original `SubmitOrder` command will be cached.
- The emulated order will be processed inside a local `MatchingCore` component.
- The `OrderEmulator` will subscribe to any needed market data (if not already) to update the matching core.
- The emulated order can be modified (by the trader) and updated (by the market) until *released* or canceled.

Released emulated orders

Once data arrival triggers / matches an emulated order locally, the following *release* actions will occur:

- The order will be transformed to either a `MARKET` or `LIMIT` order (see below table) through an additional `OrderInitialized` event.
- The orders `emulation_trigger` will be set to `NONE` (it will no longer be treated as an emulated order by any component).
- The order attached to the original `SubmitOrder` command will be sent back to the `RiskEngine` for additional checks since any modification/updates.
- If not denied, then the command will continue to the `ExecutionEngine` and on to the trading venue via an `ExecutionClient` as normal.

Order types which can be emulated

The following table lists which order types are possible to emulate, and which order type they transform to when being released for submission to the trading venue.

Order type for emulation	Can emulate	Released type
`MARKET`		n/a
`MARKET_TO_LIMIT`		n/a
`LIMIT`	✓	`MARKET`
`STOP_MARKET`	✓	`MARKET`
`STOP_LIMIT`	✓	`LIMIT`
`MARKET_IF_TOUCHED`	✓	`MARKET`
`LIMIT_IF_TOUCHED`	✓	`LIMIT`
`TRAILING_STOP_MARKET`	✓	`MARKET`
`TRAILING_STOP_LIMIT`	✓	`LIMIT`

Querying

When writing trading strategies, it may be necessary to know the state of emulated orders in the system. There are several ways to query emulation status:

Through the Cache

The following `Cache` methods are available:

- `self.cache.orders_emulated(...)`: Returns all currently emulated orders.
- `self.cache.is_order_emulated(...)`: Checks if a specific order is emulated.
- `self.cache.orders_emulated_count(...)`: Returns the count of emulated orders.

See the full API reference for additional details.

Direct order queries

You can query order objects directly using:

- `order.is_emulated`

If either of these return `False`, then the order has been *released* from the `OrderEmulator`, and so is no longer considered an emulated order (or was never an emulated order).

> ⚠ Do not hold a local reference to an emulated order. The order object transforms when the emulated order is *released*. Use the `Cache` instead.

Persistence and recovery

If a running system either crashes or shuts down with active emulated orders, then they will be reloaded inside the `OrderEmulator` from any configured cache database. This preserves order state across system restarts and recoveries.

Best practices

When working with emulated orders, consider the following best practices:

1. Always use the `Cache` for querying or tracking emulated orders rather than storing local references
2. Be aware that emulated orders transform to different types when released
3. Remember that emulated orders undergo risk checks both at submission and release

> ℹ Order emulation allows you to use advanced order types even on venues that don't natively support them, making your trading strategies more portable across different venues.

Related guides

- Events - Order events, position events, and handler dispatch.
- Execution - Order execution and fill handling.
- Positions - Positions created from order fills.
- Strategies - Order management from strategies.

‹ Execution
Previous Page

Positions ›
Next Page